Weekly Internship Report

General Information

Name: FROEHLY Jean-Baptiste

Period: Week 4 - 28/04/2025 to 02/05/2025

Weekly Objectives

- **Objective 1:** Improving the precision of attacks: efficiency and undetectability
- **Objective 2:** Further work on deepfool
- **Objective 3:** Still the idea of benchmarking various attacks

Summary

Dataset - Still the same

Special point: The dataset I was using has been removed from kaggle. So I found another one, with more images, but which were not prepared like the previous one.

Source : ADNI database - ADNI Website

New kaggle source : Dataset link - Author : Abdullah Tauseef

Infos : ~490mo, +34.0k images

This dataset provides a collection of preprocessed MRI brain scan images from the ADNI (Alzheimer's Disease Neuroimaging Initiative) project

Dataset structure

The images are arranged and classified in different categories in four directories. So I made a script to rearrange them like the previous dataset:

Pictures are referenced in a `train.csv` file and are renamed according to their diagnosis (ex: AD-0001.jpg, LMCI-2890 stand for `diagnosis_label-number_of_image`)

`train.csv`

id_code (string)	diagnosis (int)
AD-3471	4
CN-1819	0
LMCI-0760	3
...	...

Here are the different classifications and their diagnosis id

- CN - Cognitively Normal : `diagnosis=0` ; 6464 images
- EMCI - Early Mild Cognitive Impairment : `diagnosis=1` ; 9600 images
- LMCI - Late Mild Cognitive Impairment : `diagnosis=2` ; 8960 images
- AD - Alzheimer's Disease : `diagnosis=3` ; 8960 images

Attack studied

I mainly focused on **Avdersarial Attacks** and more specifically on **evasion attacks**.

General Definition : An **adversarial attack** is an attempt to manipulate a machine learning model via specifically calculated perturbations in the input data.

Evasion attacks : Evasion attacks target the model **at the time of inference**, without affecting the training. The aim is to slightly modify the input `(x)` into a version `(x')` so that the model is wrong, while keeping the modifications imperceptible.

How evasion attacks work

Principle

We seek to construct an input `(x')` such that :

`f(x') ≠ f(x) et ‖x-x'‖ < ϵ`

where `(ϵ)` is a tolerated disturbance threshold, and `(f)` is the model.

Attack scenarios

- **White box** : the attacker knows the model `(f)`, the weights `(θ)`, and the loss function `(J)`.
- **Black box** : the attacker only has access to the model's outputs.
- **Grey box** : the attacker has partial knowledge (for example, access to gradients but not to exact weights

Evasion attack types

In general, there are 4 main types of evasion attack

Fast Gradient Sign Method (FGSM)

- **Principle:** Attack based on gradient, simple but efficient
- **How it works:**
  - Calculates the gradient of the loss function with respect to the input image
  - Add a small perturbation in the direction that maximises the error
  - Formula: `x_adv = x + ε * sign(∇x J(θ, x, y))`
  - ε = 0.2 for my tests
- **Caracteristcs:**
  - One-shot attack
  - Disturbances often visible to the naked eye
  - Quick to calculate

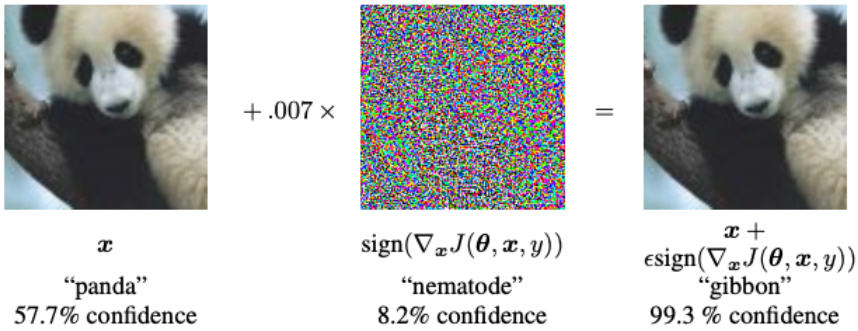


Illustration of the FGSM principle (source: TensorFlow.org) for ε = 0.007

Projected Gradient Descent (PGD)

- **Principle:** More powerful iterative version of **FGSM**
- **How it works:**
  - Apply FGSM in several small steps (10 iterations in my tests)
  - Formula: `x_adv(t+1) = Proj(x_adv(t) + α * sign(∇x J(θ, x_adv(t), y)))`
  - After each step, projection into a field ε

- **Caracteristics:**
  - More sophisticated than FGSM
  - More discrete disturbances
  - More expensive to calculate

Carlini & Wagner (C&W)

- **Principle:** Optimization-based attack designed to bypass many existing defenses
- **How it works:**
  - Defines the attack as an optimization problem: find the smallest perturbation `(δ)` such that the perturbed input is misclassified
  - Minimizes a custom loss function that balances the size of the perturbation and the likelihood of misclassification
  - Often uses L2 norm but can also be adapted to L0 or L∞
  - Formula (L2 norm): `min‖x−x′‖22 +c⋅f(x′)`
- **Characteristics:**
  - Highly effective and stealthy (very small perturbations)
  - Bypasses defensive distillation and other robust models
  - Computation-heavy due to optimization process

DeepFool

- **Principle:** Gradient-based iterative attack assuming local linearity of the model
- **How it works:**
  - Approximates the classifier as a linear model around the current input
  - At each step, computes the minimal perturbation needed to reach the closest decision boundary
  - Moves slightly toward that boundary and updates the input
  - Stops when the classifier changes its prediction
  - Formally solves: find `r` such that `argmax f(x + r) ≠ argmax f(x)`, with minimal `||r||`
- **Characteristics:**
  - Iterative and relatively fast
  - Perturbations are minimal and usually imperceptible
  - Non-targeted attack (aims to cause any misclassification)
  - Requires access to model gradients (white-box attack)
  - More precise than FGSM but computationally more expensive

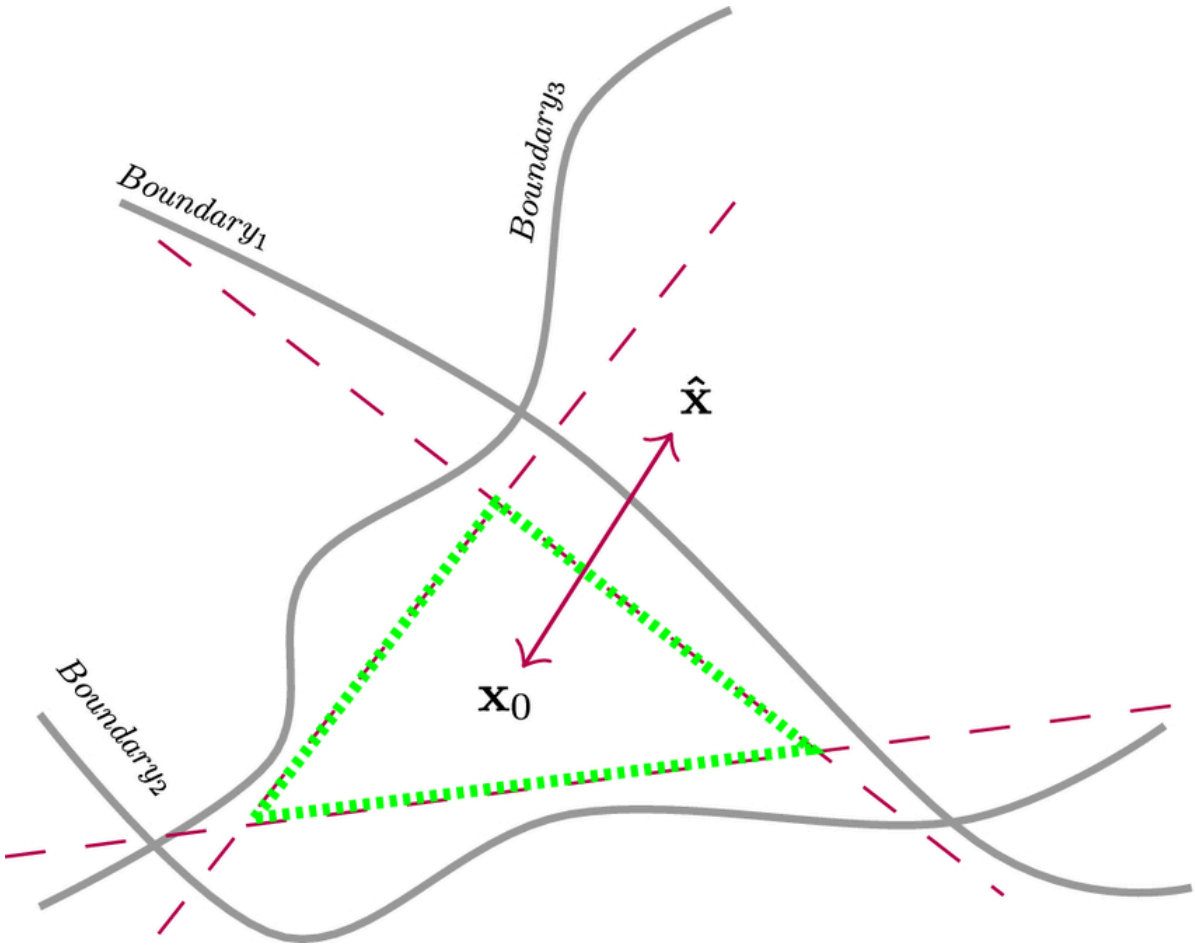


Illustration of DeepFool attack algorithm

Explanation of the Scheme

The illustration above represents the DeepFool attack algorithm in action. Here's a breakdown of the key elements:

- **Decision Boundaries (Boundary1, Boundary2, Boundary3):** The gray curved lines represent the decision boundaries of a classifier, which separate different classes in the input space. These boundaries are where the classifier's prediction changes.
- **Initial Input `x_0` :** The point `x_0` (at the center-bottom) is the original input, correctly classified by the model.
- **Perturbation (Red Arrow):** The red arrow indicates the minimal perturbation applied to `x_0` > to move it across a decision boundary. DeepFool calculates this perturbation iteratively to ensure it is as small as possible.
- **Perturbed Input `x^` :** The point `x^` is the perturbed input, now located on the other side of the decision boundary (Boundary3), resulting in misclassification by the model.
- **Search Region (Green Triangle):** The green dashed triangle represents the region where DeepFool explores to find the minimal perturbation. It iteratively adjusts the input to approach the nearest decision boundary.

This visualization highlights how DeepFool efficiently finds the smallest perturbation needed to fool the classifier, making it a powerful tool for generating adversarial examples.

Black Box attacks

- **Principle:** Attacks where the attacker has no access to model internals (weights, gradients, etc.)
- **How it works:**
  - Uses query-based approaches: observe outputs for various inputs to infer model behavior
  - Can also train a surrogate model using the outputs of the target model, and then attack the surrogate
  - Examples include Transfer Attacks, Zeroth-order Optimization (ZOO), and Boundary Attacks
- **Characteristics:**
  - More realistic in real-world scenarios
  - Usually require a large number of queries
  - Less efficient than white-box attacks but still dangerous

Common defenses

Adversarial Training, Defensive distillation, Gaussian Noise/Dropout, Gradient Smoothing, Adversarial Examples detection, randomization, etc...

Common Usages & Impacts

- **Facial Recognition**: modified glasses that can fool a system.
- **Autonomous vehicles**: disruption of road signs
- **Biometrics**: false fingerprints.

Examples

- Attack on Google Cloud Vision: a modified panda becomes a gibbon.
- Masks with special patterns to fool facial recognition

Results

Training and tests carried out locally on Nvidia RTX 4060 Laptop GPU, Python 3.10.0

20 epochs — Optimizer Adam or SGD (learning rate : 0.004, momentum : 0.9) - CNN : 2*conv+pooling, 2 fully conn., activation function: RelU

Batch size : 32

Avg Epoch Time : 90.67s

Clean Results

```
[*] Clean evaluation:
Accuracy: 0.8719
Average inference time per batch: 0.0020 seconds
Classification Report:
precision    recall  f1-score   support
0       0.97      0.99      0.98      1295
1       0.89      0.84      0.87      1947
2       0.83      0.78      0.81      1765
3       0.82      0.90      0.86      1790
accuracy                           0.87      6797
macro avg       0.88      0.88      0.88      6797
weighted avg       0.87      0.87      0.87      6797


[TIME] evaluate_model executed in 24.48 seconds
```

Attack Implementation

FGSM

```
[*] Attack: FGSM (ε=0.02)
Accuracy: 0.7138
Average attack+inference time per batch: 0.0707 seconds


Classification Report:
precision    recall  f1-score   support
0       0.93      0.86      0.90      1295
1       0.74      0.65      0.69      1947
2       0.58      0.65      0.61      1765
3       0.69      0.74      0.71      1790
accuracy                           0.71      6797
macro avg       0.74      0.73      0.73      6797
weighted avg       0.72      0.71      0.72      6797
```

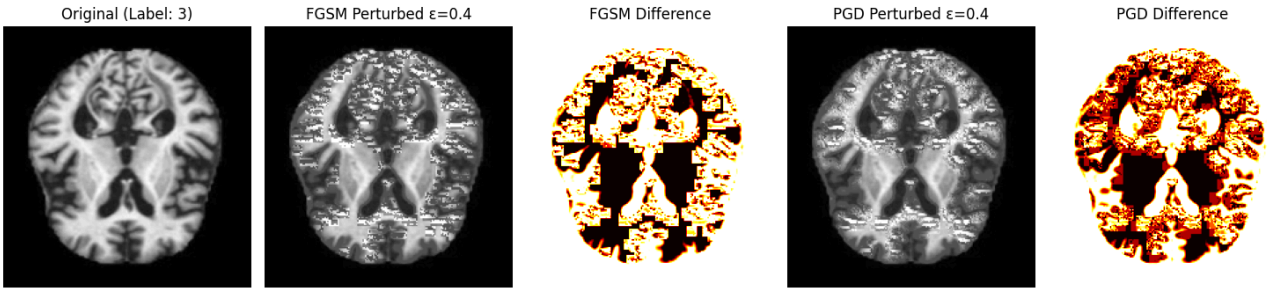
```
[TIME] test_evasion_attack executed in 29.91 seconds
```

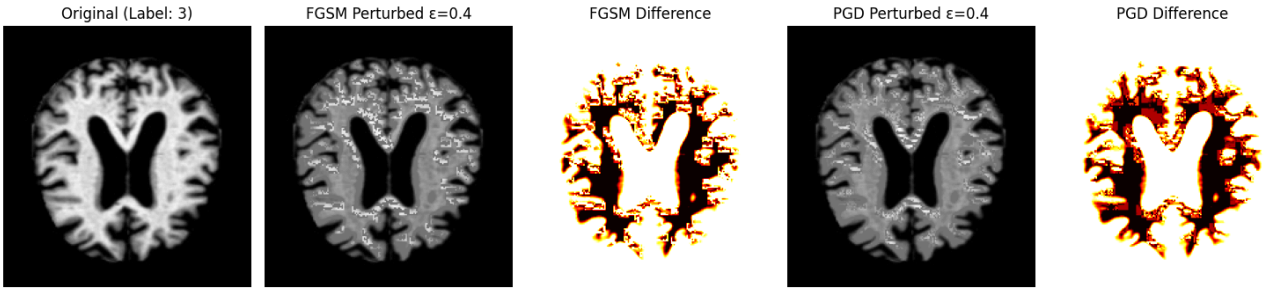
PGD

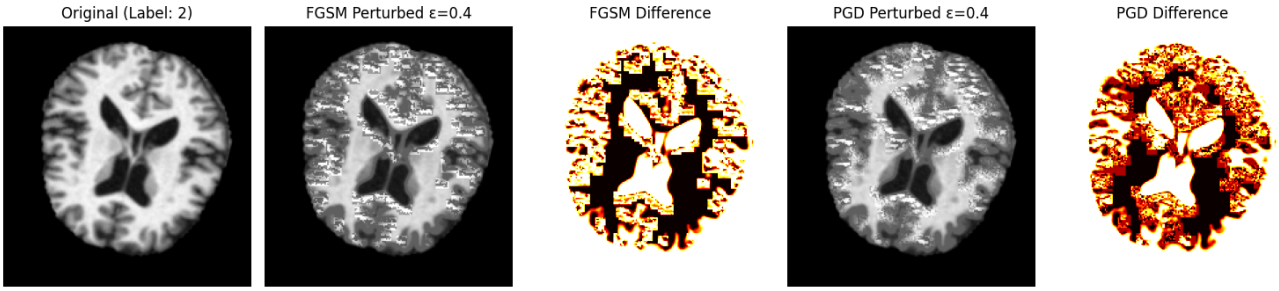
```
[*] Attack: PGD (ε=0.02, iter=10)
Accuracy: 0.6955
Average attack+inference time per batch: 0.3331 seconds

Classification Report:
precision    recall  f1-score   support
0       0.92      0.86      0.89      1295
1       0.72      0.63      0.67      1947
2       0.56      0.62      0.59      1765
3       0.67      0.72      0.69      1790
accuracy                           0.70      6797
macro avg       0.72      0.71      0.71      6797
weighted avg       0.70      0.70      0.70      6797

[TIME] test_evasion_attack executed in 82.16 seconds
```

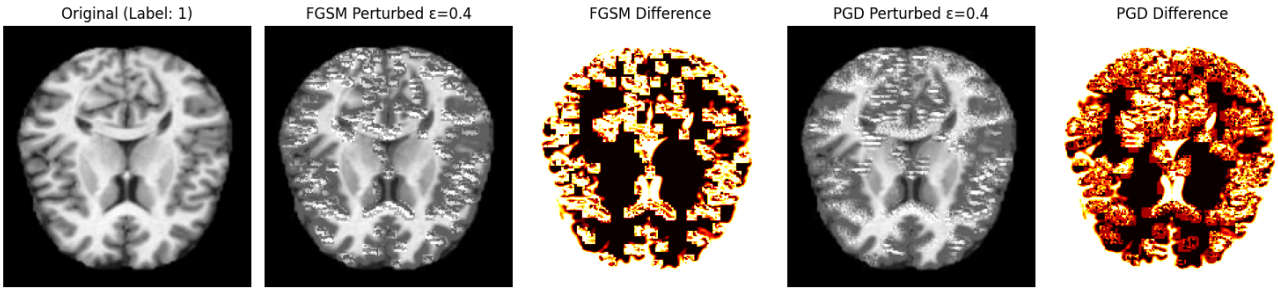
```
Accuracy Metrics:
Initial clean accuracy: 0.8719
Accuracy under FGSM attack: 0.7138 (Drop: 0.1580)
Accuracy under PGD attack: 0.6955 (Drop: 0.1764)
Accuracy under DeepFool attack: 0.1487 (Drop: 0.7231)

Performance Metrics:
Standard training time: 1906.31 seconds
Average clean inference time: 0.0020 seconds per batch
Average FGSM attack+inference time: 0.0707 seconds per batch
Average PGD attack+inference time: 0.3331 seconds per batch
Average DeepFool attack+inference time: 5.4564 seconds per batch
```

Images


Original (Label: 3)
FGSM Perturbed ε=0.4
FGSM Difference
PGD Perturbed ε=0.4
PGD Difference


Original (Label: 3)
FGSM Perturbed ε=0.4
FGSM Difference
PGD Perturbed ε=0.4
PGD Difference


Original (Label: 2)
FGSM Perturbed ε=0.4
FGSM Difference
PGD Perturbed ε=0.4
PGD Difference


Original (Label: 1)
FGSM Perturbed ε=0.4
FGSM Difference
PGD Perturbed ε=0.4
PGD Difference

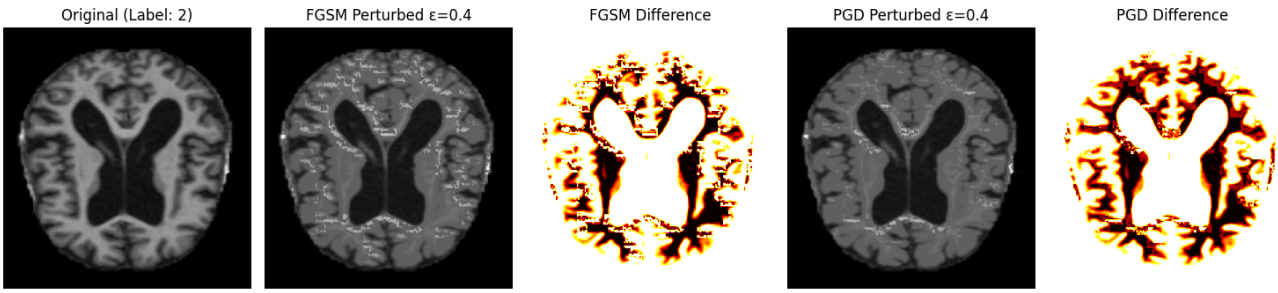
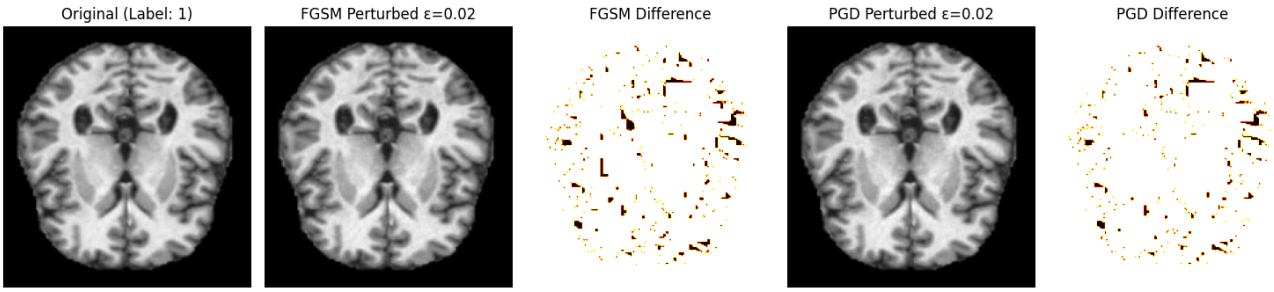
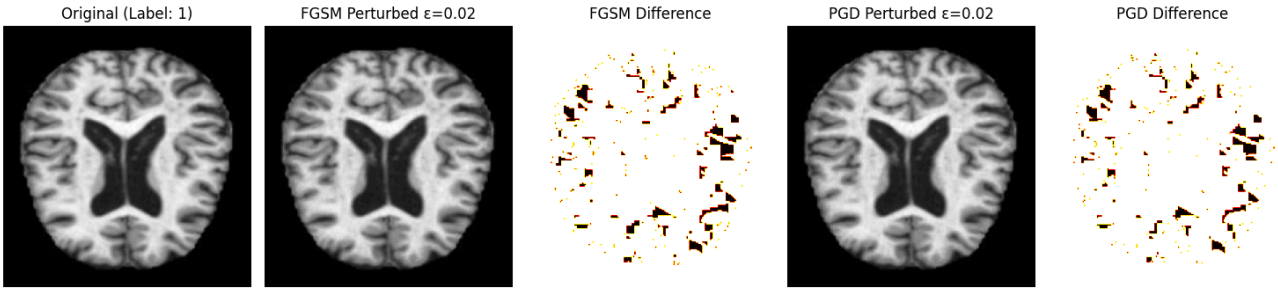
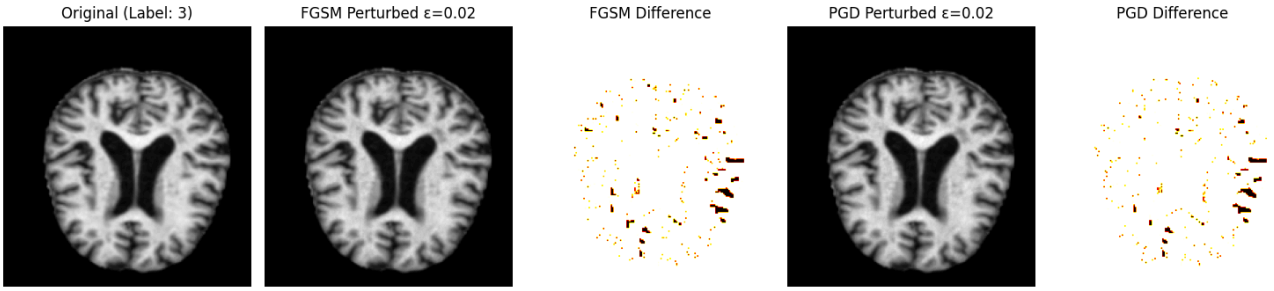
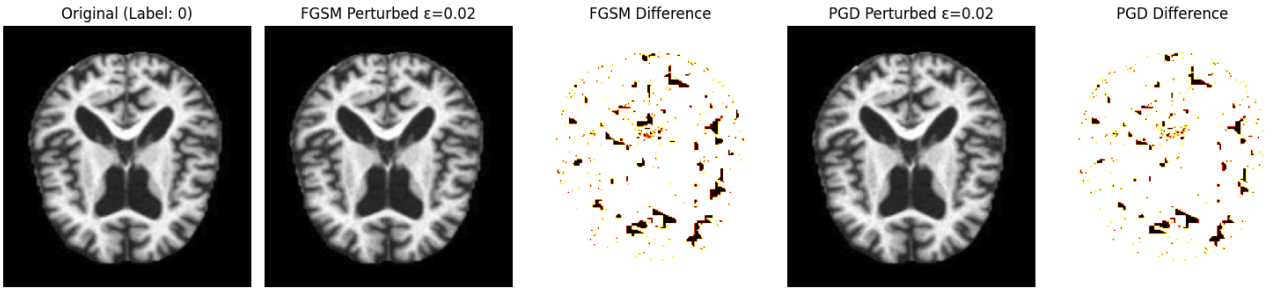

Original (Label: 2)
FGSM Perturbed ε=0.4
FGSM Difference
PGD Perturbed ε=0.4
PGD Difference

Fig.0 Attack comparizon for ε=0.4


Original (Label: 1)
FGSM Perturbed ε=0.02
FGSM Difference
PGD Perturbed ε=0.02
PGD Difference


Original (Label: 1)
FGSM Perturbed ε=0.02
FGSM Difference
PGD Perturbed ε=0.02
PGD Difference


Original (Label: 3)
FGSM Perturbed ε=0.02
FGSM Difference
PGD Perturbed ε=0.02
PGD Difference


Original (Label: 0)
FGSM Perturbed ε=0.02
FGSM Difference
PGD Perturbed ε=0.02
PGD Difference

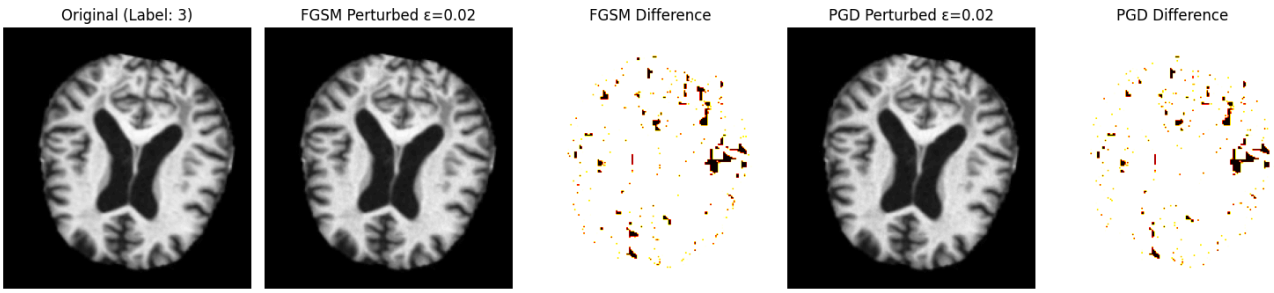

Original (Label: 3)
FGSM Perturbed ε=0.02
FGSM Difference
PGD Perturbed ε=0.02
PGD Difference

Fig.1 Attack comparison for ε=0.02

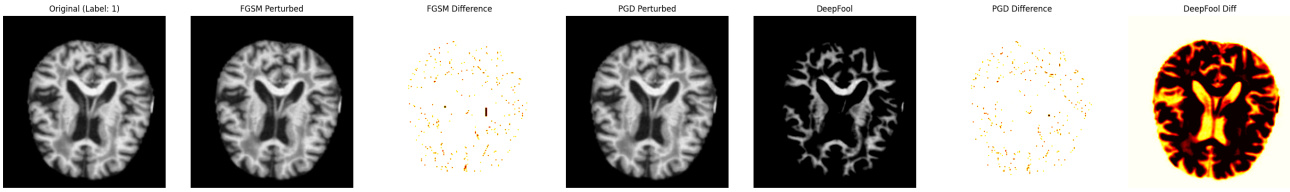


Fig.2 Attack comparison for ε=0.02 with DeepFool

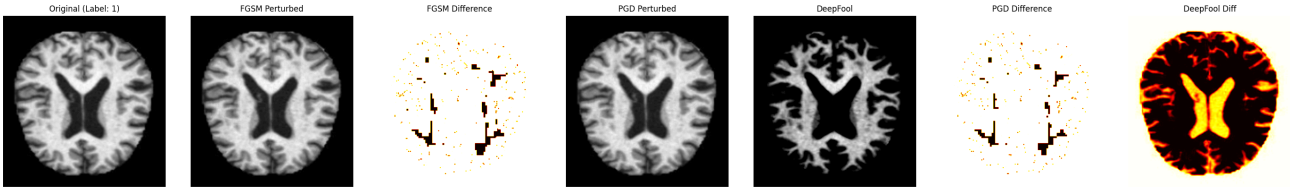


Fig.3 Attack comparison for ε=0.02 with DeepFool other example

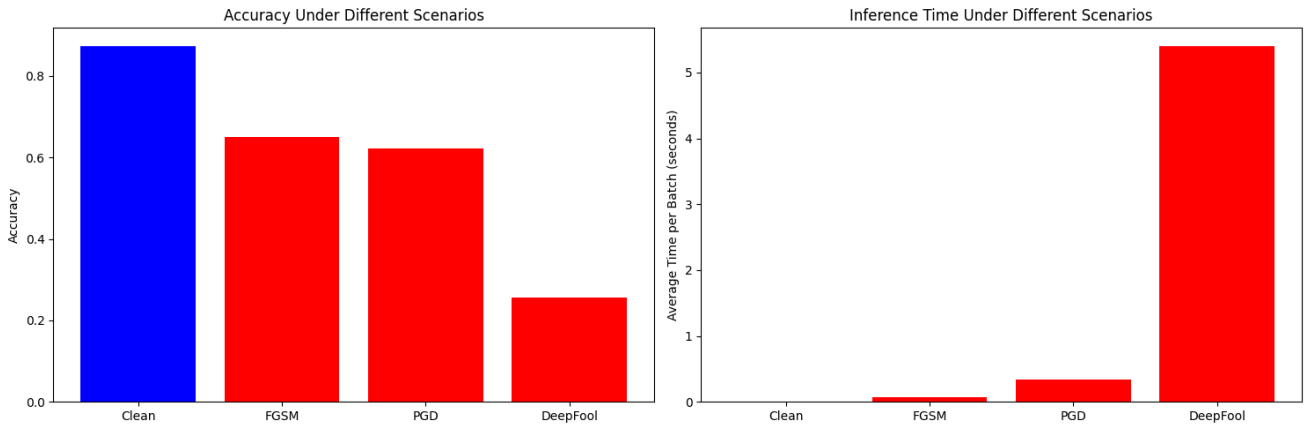


Fig.4 Accuracy comparison

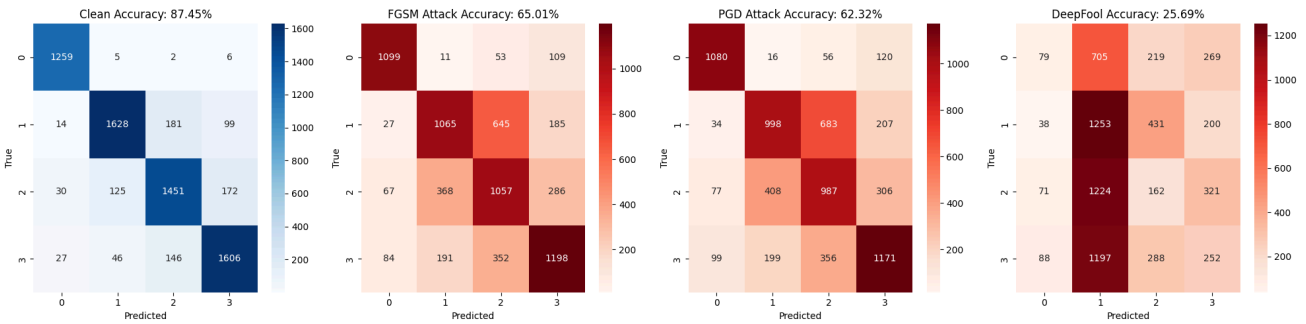


Fig.5 Confusion Matrix comparison

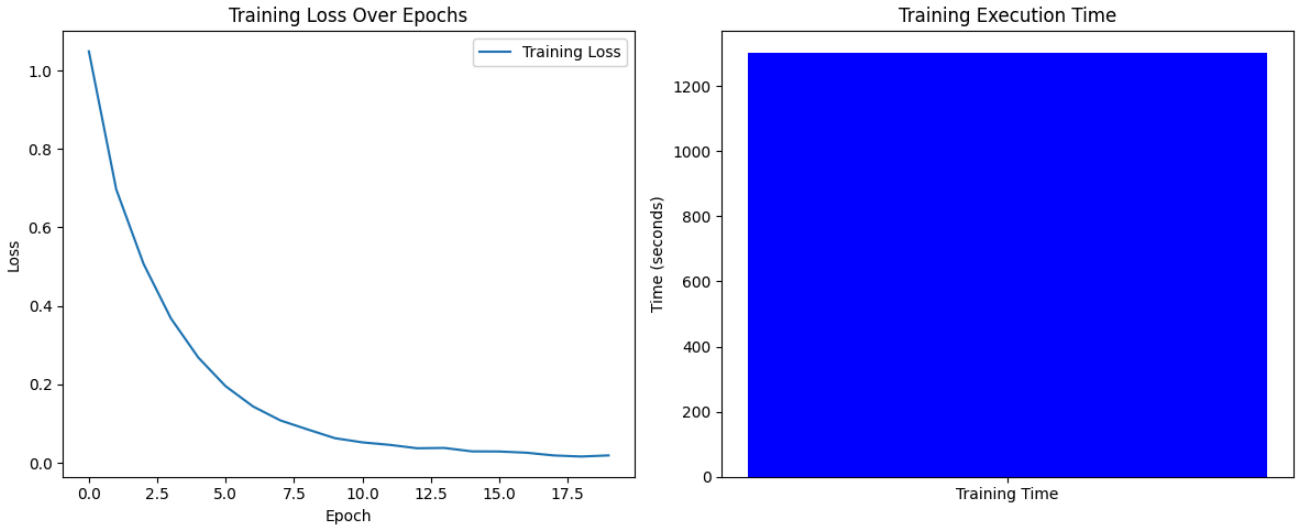


Fig.6 Training Metrics (20 epochs)

Analysis and Interpretation

We can see that it's very easy to obtain invisble perturbation for Human eye with FGSM and PGD with an interessant efficiency of attack.

We can also see that DeepFool, at least with the parameters used so far, is not as discreet as expected on grayscale images of this type. It's undeniably extremely effective, but the difference is easily distinguishable

Initial Model Performance

Clean Data

- Accuracy: 87.19%
- Excellent performance on class 0 (f1-score = 0.98)
- Balanced results across other classes with f1-scores between 0.81 and 0.87
- Macro average f1-score: 0.88 → indicates good generalization across classes
- Fast inference: 0.0020 seconds per batch

Impact of Adversarial Attacks

FGSM (ε = 0.02)

- Accuracy: 71.38% → **drop of 15.80 points**
- Significant decline in performance on classes 1–3 (f1-scores range from 0.61 to 0.71)
- Class 0 remains relatively robust (f1-score = 0.90)
- Attack + inference time: 0.0707 seconds per batch (×35 slower than clean)

PGD (ε = 0.02, 10 iterations)

- Accuracy: 69.55% → **drop of 17.64 points**
- More impactful than FGSM, especially on classes 1 and 2
- f1-scores drop further (as low as 0.59 for class 2)
- Attack + inference time: 0.3331 seconds per batch (×166 slower than clean)

DeepFool

- Accuracy: 14.87% → **drop of 72.31 points**
- Catastrophic degradation — the model fails to make meaningful predictions
- Highlights extreme sensitivity to subtle, well-crafted perturbations
- Attack + inference time: 5.4564 seconds per batch (over ×2700 slower than clean inference)

Conclusion

- The model performs very well on clean data, with high and balanced accuracy
- However, it is vulnerable to even low-magnitude adversarial attacks (ε = 0.02)
- PGD and especially DeepFool demonstrate how easily the model's performance can collapse
- These results emphasize the necessity of integrating adversarial robustness into model development pipelines

Difficulties Encountered

- **Problem 1:** I had problems with C&W : the execution takes far too long, I've never been able to afford to finish one, so either I have to completely revise the parameters, or I have to abandon the idea of implementing this attack.
- **Problem 2:** DeepFool doesn't behave very well on my images, so I think I need to review my settings in detail to try other things. I'm not sure whether it's a question of the dataset or the parameters, given that on paper, according to the maths behind it, there's no reason why you can't get interesting and effective results.

FROEHLY Jean-Baptiste, Friday 02/05/2025